Git & GitHub – Erste Schritte

Anleitung für unser Testrepo **ingenieur-toolbox**

Worum geht's? Wir arbeiten im Softwareprojekt gemeinsam an Code. Damit das nicht über USB-Sticks und Mails mit „final_v3_wirklich_final.py“ läuft, nutzen wir **Git** (Versionsverwaltung auf deinem Rechner) und **GitHub** (die Online-Plattform, auf der das gemeinsame Repo liegt). Mit dem Testrepo übst du den kompletten Ablauf einmal gefahrlos – kaputt machen kannst du nichts.

Ablauf im Überblick

1	Einrichten	VS Code, Python, Git installieren, GitHub-Account anlegen
2	Klonen	Repo von GitHub auf den eigenen Rechner holen
3	Übung 1	Direkt auf main arbeiten: Namen eintragen, pull, commit, push, Merge-Konflikte lösen
4	Übung 2	Eigenen Branch anlegen, eigenes Werkzeug programmieren (selbst oder mit KI)
5	Pull Request	Den Branch auf GitHub zur Übernahme in main vorschlagen

Zeitaufwand: ca. 30–45 Minuten beim ersten Mal.

Kleines Wörterbuch

Repository (Repo)	Projektordner, dessen komplette Änderungsgeschichte Git speichert
Clone	Einmaliges Herunterladen eines Repos von GitHub
Commit	Ein gespeicherter Zwischenstand mit kurzer Beschreibung
Push / Pull	Commits zu GitHub hochladen / neue Commits von GitHub holen
Branch	Eigener Arbeitszweig – du arbeitest, ohne main zu stören
main	Der Hauptzweig. Hier liegt nur geprüfter, funktionierender Code
Merge-Konflikt	Zwei Leute haben dieselbe Stelle geändert – Git fragt, welche Version bleiben soll
Pull Request (PR)	Antrag auf GitHub: „Bitte meinen Branch in main übernehmen“

1. Einrichten (einmalig)

1.1 Visual Studio Code installieren

- Download: **https://code.visualstudio.com** – Version für dein Betriebssystem wählen und installieren.
- Windows: Bei der Installation die Häkchen *„Zu PATH hinzufügen“* und *„Mit Code öffnen“* setzen.
- VS Code starten, links auf das Erweiterungen-Symbol (vier Quadrate) klicken, nach **Python** (von Microsoft) suchen und installieren.
- Optional: Sprachpaket **German Language Pack** – die Anleitung nennt die Menüs auf Deutsch und Englisch.

1.2 Python installieren

- Download: **https://www.python.org/downloads** (aktuelle Version 3.x).
- Windows: Im ersten Installationsfenster unbedingt **„Add python.exe to PATH“** anhaken!
- macOS/Linux: Python ist meist schon da. Prüfen mit `python3 --version`.

1.3 Git installieren

- Windows: **https://git-scm.com/downloads** – Installer ausführen, alle Standardeinstellungen übernehmen (einfach „Next“ klicken).
- macOS: Im Terminal `git --version` eingeben – macOS bietet dann die Installation an.
- Linux: `sudo apt install git` (Ubuntu/Debian).

Danach VS Code **neu starten** und ein Terminal öffnen: Menü **Terminal > Neues Terminal** (*Terminal* > *New Terminal*). Prüfen, ob alles da ist:

```
git --version
python --version        (Windows evtl.: py --version, macOS/Linux: python3 --version)
```

1.4 Git sagen, wer du bist

Jeder Commit wird mit Name und E-Mail gespeichert. Einmalig im Terminal eingeben – **dieselbe E-Mail wie bei GitHub** verwenden:

```
git config --global user.name "Max Mustermann"
git config --global user.email "max.mustermann@beispiel.de"
git config --global pull.rebase false
git config --global core.editor "code --wait"
```

Die letzten beiden Zeilen sorgen dafür, dass `git pull` ohne Rückfrage funktioniert und Git zum Bearbeiten von Nachrichten VS Code öffnet (statt eines Terminal-Editors, aus dem man nicht mehr herauskommt).

1.5 GitHub-Account anlegen

- Auf **https://github.com** > **Sign up**. E-Mail, Passwort und einen Benutzernamen wählen (am besten erkennbar, z. B. *max-mustermann*).
- E-Mail-Adresse bestätigen. Empfohlen: Zwei-Faktor-Authentifizierung aktivieren (Settings > Password and authentication).
- **Deinen GitHub-Benutzernamen an den Projektleiter schicken.** Du bekommst dann eine Einladung per Mail für das Repo – diese **annehmen**, sonst darfst du später nicht pushen.

2. Repo klonen (herunterladen)

Die Repo-Adresse findest du auf der GitHub-Seite des Repos über den grünen Button **<> Code** (HTTPS-Link kopieren). Sie sieht so aus:

```
https://github.com/<BENUTZERNAME>/ingenieur-toolbox.git
```

Variante A – mit VS Code (einfach):

- VS Code öffnen, links das Symbol **Quellcodeverwaltung** (*Source Control*, drei Punkte mit Linien) anklicken > **Repository klonen** (*Clone Repository*).
- Link einfügen, Enter, einen Speicherort wählen (z. B. *Dokumente/Projekte*).
- Beim ersten Mal öffnet sich der Browser zum Anmelden bei GitHub – erlauben.
- Auf **Öffnen** klicken. Fertig – du siehst links die Dateien.

Variante B – mit dem Terminal:

```
cd Dokumente
git clone https://github.com/<BENUTZERNAME>/ingenieur-toolbox.git
cd ingenieur-toolbox
code .                  (öffnet den Ordner in VS Code)
```

Programm testen – im VS-Code-Terminal (es muss im Ordner *ingenieur-toolbox* stehen):

```
python main.py          (Windows evtl.: py main.py / macOS, Linux: python3 main.py)
```

Es erscheint ein Menü mit Werkzeugen (Ohmsches Gesetz, Kreisfläche, …). Mit 0 beenden.

3. Übung 1: Direkt auf main arbeiten

Zum Eingewöhnen arbeiten alle direkt auf dem Hauptzweig **main**. Ziel: den Grundablauf verinnerlichen – und einmal einen Merge-Konflikt erleben und lösen. Der Ablauf ist **immer** derselbe:

1	**`git pull`**	Neuesten Stand holen – **bevor** du anfängst
2	**`ändern`**	Datei bearbeiten und speichern, ggf. testen
3	**`git add`**	Geänderte Datei für den Commit vormerken
4	**`git commit`**	Zwischenstand mit Nachricht speichern
5	**`git pull`**	Nochmal holen – vielleicht war jemand schneller
6	**`git push`**	Hochladen

3.1 Aufgabe: Trag dich ins Team ein

Öffne die Datei `TEILNEHMER.md` und schreib in eine **neue Zeile ganz unten** deinen Namen und dein Lieblingsfach, z. B. - *Max Mustermann – Technische Mechanik*. Speichern (Strg+S).

Variante A – Terminal:

```
git pull
   ... TEILNEHMER.md bearbeiten und speichern ...
git status                      (zeigt: TEILNEHMER.md geändert)
git add TEILNEHMER.md
git commit -m "Max Mustermann ins Team eingetragen"
git pull
git push
```

Variante B – VS Code:

- Links **Quellcodeverwaltung** öffnen. Geänderte Dateien stehen unter *Änderungen* (*Changes*).

- Beim Überfahren der Datei auf das **+** klicken (= `git add`).
- Oben eine Commit-Nachricht eintippen und auf **Commit** klicken.
- Auf **Änderungen synchronisieren** (*Sync Changes*) klicken – das macht pull und push in einem.

Gute Commit-Nachricht: kurz, sagt *was* passiert ist. Gut: *„Temperatur-Umrechnung hinzugefügt“*. Schlecht: *„update“, „asdf“, „fix“*.

3.2 Push abgelehnt? Ganz normal!

Wenn jemand vor dir gepusht hat, lehnt GitHub deinen Push ab (*„rejected … fetch first“*). Kein Problem: `git pull`, dann wieder `git push`. Hat der andere eine **andere** Stelle geändert, fügt Git beides automatisch zusammen. Öffnet sich dabei ein Fenster für eine Merge-Nachricht: einfach schließen.

3.3 Merge-Konflikt lösen

Da alle in die letzte Zeile derselben Datei schreiben, wird es bei einigen **gewollt** knallen. Git meldet beim Pull: *„CONFLICT (content): Merge conflict in TEILNEHMER.md“*. In der Datei steht dann:

```
<<<<<<< HEAD
- Max Mustermann - Technische Mechanik       (deine Version)
=======
- Erika Musterfrau - Elektrotechnik          (Version von GitHub)
>>>>>>> 3f2a1c9...
```

- VS Code bietet über dem Block Knöpfe an: *Accept Current* (deine), *Accept Incoming* (die andere), **Accept Both** (beide). Hier ist **Accept Both** richtig – beide Namen sollen bleiben.
- Oder von Hand: die Markierungszeilen <<<<<<<, ======= und >>>>>>> löschen, so dass das Ergebnis stimmt.
- Datei speichern, dann abschließen:

```
git add TEILNEHMER.md
git commit -m "Merge-Konflikt in TEILNEHMER.md gelöst"
git push
```

Faustregel: Ein Konflikt ist kein Fehler, sondern eine Frage von Git: „Welche Version soll bleiben?“ Bei Code immer überlegen, nicht blind klicken – und im Zweifel mit dem Kollegen reden, dessen Änderung betroffen ist. Notausgang: `git merge --abort` setzt alles auf den Stand vor dem Pull zurück.

4. Übung 2: Eigener Branch mit eigenem Werkzeug

Im echten Projekt soll **main** immer funktionieren. Deshalb arbeitet jeder auf einem eigenen **Branch** (Arbeitszweig): Du kannst dort ausprobieren, committen und pushen, ohne dass andere etwas davon merken. Erst wenn es fertig ist, wird es per **Pull Request** in main übernommen.

4.1 Branch anlegen

Name-Schema: `feature/<vorname>-<thema>`

```
git switch main         (sicherstellen, dass du auf main bist)
git pull                (neuesten Stand holen)
git switch -c feature/max-biegemoment
```

In VS Code: unten links in der Statusleiste steht der aktuelle Branch. Ein Klick darauf > **Neuen Branch erstellen** (*Create new branch*) geht auch.

4.2 Eigenes Werkzeug programmieren

Jedes Werkzeug ist **eine eigene Datei** im Ordner *werkzeuge/*. Neue Dateien erscheinen automatisch im Menü – ihr kommt euch also nicht in die Quere.

- `werkzeuge/_vorlage.py` kopieren und umbenennen, z. B. in `biegemoment.py` – **ohne** Unterstrich am Anfang, keine Umlaute/Leerzeichen.
- `NAME`, `BESCHREIBUNG` und `ausfuehren()` anpassen.
- Mit `python main.py` testen, bis es funktioniert.
- Ideen: Biegemoment, Leistung P = U·I, Drehmoment, Temperatur- oder Druck-Umrechnung, Reihen-/Parallelschaltung … (siehe README.md). Sprecht kurz ab, wer was macht.

Mit KI (ChatGPT, Copilot, Claude …): Gib der KI die Vorlage und eine klare Aufgabe, z. B.:

```
Hier ist eine Vorlage für ein Python-Werkzeug (Inhalt von _vorlage.py einfügen).
Schreibe nach diesem Muster ein Werkzeug "Biegemoment", das für einen
Einfeldträger mit mittiger Einzellast F und Länge L das maximale
Biegemoment M = F*L/4 berechnet. Nutze zahl_eingeben() für die Eingaben.
```

Wichtig bei KI-Code: Du committest ihn unter deinem Namen – also verstehen, testen und Ergebnisse einmal von Hand nachrechnen. Niemals Passwörter, Kundendaten oder internen Projektcode in öffentliche KI-Tools kopieren.

4.3 Commit & Push des Branches

```
git add werkzeuge/biegemoment.py
git commit -m "Werkzeug Biegemoment hinzugefügt"
git push -u origin feature/max-biegemoment
```

Das `-u origin` … braucht es nur beim ersten Push eines neuen Branches, danach reicht `git push`. In VS Code heißt der Knopf **Branch veröffentlichen** (*Publish Branch*).

4.4 Pull Request stellen

- Auf GitHub erscheint ein gelber Hinweis mit **Compare & pull request** – anklicken (alternativ: Reiter *Pull requests* > *New pull request*).
- Base: **main**, Compare: **dein Branch**. Titel und 1–2 Sätze, was du gemacht hast.
- **Create pull request**. Ein Teamkollege schaut drüber (Review) und klickt auf **Merge**.
- Danach lokal zurück auf main und den neuen Stand holen:

```
git switch main
git pull
```

Geschafft – genau so arbeiten wir im echten Projekt. Für die nächste Änderung wieder bei 4.1 starten.

Wenn es mal hakt

- **„permission denied“ / 403:** Einladung zum Repo noch nicht angenommen (Mail prüfen).
- **„python“ wird nicht gefunden:** `py` bzw. `python3` probieren, oder Python mit Häkchen „Add to PATH“ neu installieren.
- **„Your local changes would be overwritten“ beim Pull:** Erst committen, dann pullen.
- **Völlig verheddert?** Nicht wild Befehle ausprobieren – Bescheid sagen, das ist in 2 Minuten gelöst.

5. Das bitte NICHT machen

Nicht	Warum
X Pushen ohne vorher zu pullen	Führt zu abgelehntem Push und unnötigen Konflikten. Immer erst git pull.
X Konflikte blind wegklicken	Nicht einfach „meine Version“ wählen – sonst ist die Arbeit der anderen weg.
X Ab Übung 2: direkt auf main pushen	Dann gilt: main muss immer laufen. Änderungen per Branch + Pull Request.
X git push --force	Überschreibt die Arbeit anderer unwiderruflich. Nur nach Absprache.
X Passwörter, API-Keys, .env committen	Bleiben in der Historie für immer sichtbar – auch nach dem Löschen.
X Große/generierte Dateien committen	Keine venv-Ordner, Build-Ausgaben, Videos, ZIPs. Dafür gibt's die .gitignore.
X Kaputten / ungetesteten Code pushen	Vor jedem Commit einmal laufen lassen.
X Riesen-Commits nach 2 Wochen	Lieber klein und oft committen – ein Thema pro Commit.
X Nichtssagende Commit-Messages	„update“ hilft in drei Monaten niemandem.
X Dateikopien wie code_alt.py	Git ist das Backup. Alte Stände holt man über die Historie zurück.
X KI-Code blind übernehmen	Erst verstehen und testen, dann committen.

Spickzettel

```
git status                        Was hat sich geändert?
git pull                          Neuesten Stand holen
git switch -c feature/name-thema  Neuen Branch anlegen und wechseln
git switch main                   Zurück auf main
git merge --abort                 Missglückten Merge abbrechen
git add <datei>                   Datei für Commit vormerken
git commit -m "Nachricht"         Zwischenstand speichern
git push                          Hochladen
git log --oneline                 Historie anzeigen
```